

Jarvis: Implementación y Evaluación Empírica de un Asistente de Lenguaje Local sin GPU con Capacidades RAG, Tool Calling y Evaluación Automatizada

Dilan Andrés Quisilema Guamán

Ingeniería en Sistemas

Machine Learning

Ecuador, 2026

diquisilemagu@uide.edu.ec

Abstract—Este trabajo presenta el diseño, implementación y evaluación empírica de Jarvis, un asistente conversacional basado en modelos de lenguaje de gran escala (LLM) que opera completamente en hardware de consumo sin GPU dedicada. El sistema integra inferencia local mediante Ollama con el modelo Llama 3.2 3B en tres niveles de cuantización, una canalización de Generación Aumentada por Recuperación (RAG) sobre un corpus literario en español del siglo XVII, y seis herramientas externas conectadas vía llamada a función equivalente al protocolo MCP. Los experimentos muestran que la cuantización Q4_K_M constituye el punto óptimo entre velocidad (27.87 tok/s) y calidad (2.6/3.0) en una máquina con 7.9 GB de RAM disponible. La evaluación automatizada sobre 20 indicaciones categorizadas reporta una tasa de éxito global del 65%, con un 100% en tareas de herramienta individual y un 25% en tareas de razonamiento encadenado, evidenciando los límites reales de los modelos pequeños en hardware restringido.

Index Terms—LLM local, cuantización, RAG, tool calling, evaluación empírica, Ollama, inferencia CPU

I. ARQUITECTURA DEL SISTEMA

Jarvis está compuesto por cuatro módulos principales que operan de forma coordinada. El orquestador central (`jarvis_agent.py`) recibe la consulta del usuario en lenguaje natural y determina mediante una primera llamada al modelo si la tarea requiere el uso de una herramienta externa. El modelo responde exclusivamente con un objeto JSON del tipo `{"tool": "nombre", "parametros": {...}}`, que el orquestador parsea con un mecanismo de extracción robusto tolerante a texto adicional alrededor del JSON. Si la herramienta indicada existe en el registro, se ejecuta la función Python correspondiente y los datos reales se inyectan en un segundo prompt para que el modelo genere una respuesta en lenguaje natural. Este flujo de dos etapas permite separar la decisión de usar una herramienta de la generación de la respuesta final, lo que mejora la tasa de éxito frente a un enfoque de llamada única donde el modelo debe razonar y formatear simultáneamente.

El módulo RAG (`rag_pipeline.py`) implementa una canalización de recuperación sobre un índice vectorial persistente construido con ChromaDB. Los

fragmentos del corpus se codifican mediante el modelo `paraphrase-multilingual-MiniLM-L12-v2` de Sentence Transformers, seleccionado por su capacidad de manejar español del siglo XVII con mayor fidelidad semántica que alternativas entrenadas exclusivamente en inglés moderno. En cada consulta se recuperan los cinco fragmentos más similares por similitud coseno y se concatenan como contexto en el prompt. El prompt del modelo incluye instrucciones explícitas para interpretar el vocabulario arcaico y responder en español contemporáneo, lo que resultó determinante para la calidad final de las respuestas con RAG.

El módulo de herramientas (`herramientas.py`) registra seis funciones externas organizadas en dos grupos. El primer grupo comprende tres APIs públicas sin autenticación: clima actual mediante Open-Meteo con cobertura de más de 25 ciudades latinoamericanas y europeas, precio de criptomonedas mediante CoinGecko con soporte para múltiples monedas fiat, e información de repositorios mediante la API pública de GitHub con límite de 60 solicitudes por hora. El segundo grupo comprende tres funciones de Google Calendar autenticadas vía OAuth 2.0 que consultan simultáneamente los cuatro calendarios vinculados a la cuenta del usuario: personal, universitario Canvas, Mundial 2026 y festivos de Ecuador. La corrección de zona horaria fue un aspecto crítico de la implementación: todas las consultas se convierten a UTC antes de enviarse a la API de Google y los resultados se reconvierten a hora local de Ecuador (UTC-5) antes de presentarse al modelo.

La inferencia se delega íntegramente a Ollama, que expone el modelo en `localhost:11434` con los pesos cuantizados cargados en RAM del sistema. Este diseño desacopla completamente la lógica de la aplicación de los detalles de inferencia de bajo nivel y permite cambiar de modelo con un solo parámetro de configuración sin modificar ningún otro componente del sistema.

II. METODOLOGÍA

A. Hardware y entorno de prueba

Todos los experimentos se ejecutaron en un portátil con procesador Intel Core i5-13420H (8 núcleos físicos, 12 hilos lógicos, frecuencia base de 2.1 GHz y frecuencia turbo de hasta 4.6 GHz), 16 GB de RAM DDR5 a 5200 MT/s configurados en modo dual channel, y sin GPU dedicada. La memoria RAM disponible en reposo fue de aproximadamente 7.9 GB, medida inmediatamente antes de cada experimento para garantizar condiciones iniciales consistentes. El sistema operativo es Windows 11 con Ollama ejecutándose como servicio en segundo plano. La restricción de hardware sin GPU es intencional: fuerza a que cada decisión técnica tenga consecuencias medibles y directamente observables en las métricas de velocidad, memoria y calidad.

La medición de RAM se realizó leyendo el campo *Working Set* del proceso `llama-server.exe` en el Monitor de Recursos de Windows durante la inferencia activa, dado que la librería `psutil` de Python no refleja con precisión la memoria del proceso hijo que Ollama crea para el modelo. Esta distinción es importante porque los valores difieren hasta en un orden de magnitud entre ambos métodos de medición.

B. Modelos y niveles de cuantización

Se seleccionó el modelo base `llama3.2:3b-instruct` por ser el modelo de mayor calidad que cabe en la memoria disponible con margen operativo suficiente para el contexto y las capas de atención. Se evaluaron tres niveles de cuantización del formato GGUF: Q8_0 (8 bits uniformes por peso), Q4_K_M (4 bits con agrupación de bloques y cuantización mixta) y Q3_K_M (3 bits con agrupación de bloques). La cuantización reduce la representación numérica de los pesos del modelo, disminuyendo el tamaño en disco y el consumo de RAM a costa de cierta pérdida de precisión que se manifiesta de forma heterogénea según el tipo de tarea.

C. Corpus RAG

Se utilizó el texto de *Don Quijote de la Mancha* de Miguel de Cervantes, obtenido del Proyecto Gutenberg bajo licencia de dominio público. El corpus fue limitado a los primeros 160,103 caracteres del texto literario desde el Capítulo Primero, omitiendo el encabezado legal del archivo fuente. Esto corresponde aproximadamente a los ocho primeros capítulos de la obra. El recorte fue necesario dado que la indexación del texto completo (2.1 millones de caracteres) generaba un proceso de embedding que superaba los tiempos prácticos en hardware CPU. El texto se dividió en 178 fragmentos de 1,000 caracteres con solapamiento de 100 caracteres entre fragmentos consecutivos para evitar la pérdida de información en los bordes de corte.

D. Conjunto de prueba y criterios de evaluación

Se diseñó un conjunto de 20 indicaciones distribuidas en cinco categorías: chat puro (4 indicaciones), RAG requerido (4), herramienta requerida (4), multistep o encadenado (4) y adversarial o fuera de alcance (4). Las indicaciones fueron

redactadas en español coloquial para evaluar el comportamiento del modelo ante el tipo de consultas que un usuario real produciría, evitando el sesgo de prompts artificialmente bien estructurados. Cada indicación declara criterios de éxito objetivos y verificables automáticamente: presencia de palabras clave esperadas en la respuesta, nombre de la herramienta esperada a invocar, longitud mínima de respuesta en palabras y palabras cuya presencia en la respuesta indica alucinación. El evaluador automático clasifica cada resultado en tres niveles: éxito, parcial o fallo, y registra latencia en segundos y tokens consumidos por indicación.

III. RESULTADOS

A. Parte A: Estudio de cuantización

La Tabla I resume las métricas medidas para cada nivel de cuantización. Cada medición de tokens por segundo es el promedio de tres repeticiones con el mismo prompt de 200 tokens para compensar la variabilidad del warm-up de la primera inferencia.

TABLE I
COMPARATIVA DE NIVELES DE CUANTIZACIÓN — LLAMA 3.2 3B

Modelo	Tamaño	RAM	Tok/s	Calidad
Q8_0	3.19 GB	7.30 GB	4.95	2.2/3.0
Q4_K_M	1.88 GB	4.72 GB	27.87	2.6/3.0
Q3_K_M	1.57 GB	4.10 GB	6.73	2.0/3.0

Q4_K_M resultó ganador en los tres frentes simultáneamente. Q8_0 consume 7.30 GB de RAM, equivalente al 92% de la memoria disponible, pero genera texto a solo 4.95 tok/s porque el mayor ancho de los pesos satura el bus de memoria entre la RAM y la CPU. Q3_K_M libera algo más de memoria pero introduce alucinaciones factuales concretas: en la pregunta sobre el paper *Attention is All You Need*, el modelo inventó el nombre de autor “Jonas Winters”, que no existe en la literatura. La Tabla II desglosa las puntuaciones por tipo de tarea para los tres modelos.

TABLE II
PUNTUACIONES DE CALIDAD POR TAREA Y CUANTIZACIÓN (0 A 3)

Tarea	Q8	Q4	Q3
Matemáticas	3	3	3
Código	2	2	2
Resumen	3	3	3
Hecho histórico	2	3	1
Razonamiento	1	2	1
Promedio	2.2	2.6	2.0

El razonamiento encadenado y los hechos históricos precisos son las categorías más sensibles a la cuantización. El resumen y las matemáticas básicas son robustos incluso a 3 bits. Las Figuras 1 y 2 muestran las relaciones entre las tres variables medidas.

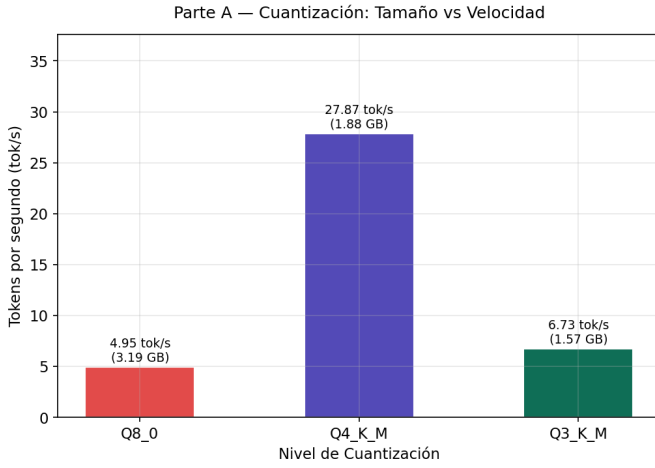


Fig. 1. Tamaño en disco vs velocidad de inferencia por nivel de cuantización.

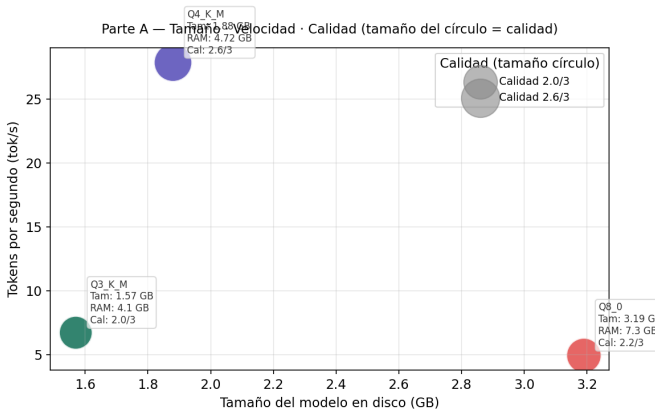


Fig. 2. Relación simultánea entre tamaño, velocidad y calidad. El tamaño del círculo representa la puntuación de calidad promedio.

B. Parte B: Experimento de caché KV

La Tabla III registra el comportamiento del modelo Q4_K_M ante cuatro longitudes de contexto distintas. El prompt de prueba consiste en texto técnico en inglés sobre sistemas de caché, lo que garantiza que el modelo lo procese de forma activa y no lo descarte.

TABLE III
IMPACTO DEL TAMAÑO DE CONTEXTO EN VELOCIDAD Y TIEMPO — Q4_K_M

Contexto	Tok/s	Tiempo	Estado
512 tokens	3.81	29.4 s	Normal
2048 tokens	2.59	38.7 s	Normal
8192 tokens	5.60	18.4 s	Caché reutilizada
16384 tokens	0.30	358.2 s	Swap activado

El resultado más notable es el del contexto de 8,192 tokens, que superó en velocidad al de 2,048. Esto ocurrió porque Ollama detectó que el inicio del prompt era idéntico al prompt anterior y reutilizó los vectores clave y valor ya calculados en memoria, evitando recomputar la atención sobre toda la

secuencia previa. Este comportamiento es el mecanismo de caché KV funcionando exactamente como se diseñó: los costos de cómputo de la atención sobre el contexto histórico se pagan una sola vez. A 16,384 tokens el sistema excedió la RAM disponible y activó el mecanismo de swap del sistema operativo, utilizando el disco como extensión de la memoria. Dado que el ancho de banda de un SSD es aproximadamente 20 veces menor al de la RAM, la velocidad cayó de 5.60 a 0.30 tok/s y el tiempo de respuesta ascendió a 358 segundos, confirmando empíricamente el límite práctico de esta configuración de hardware. Las Figuras 3 y 4 ilustran estos patrones.

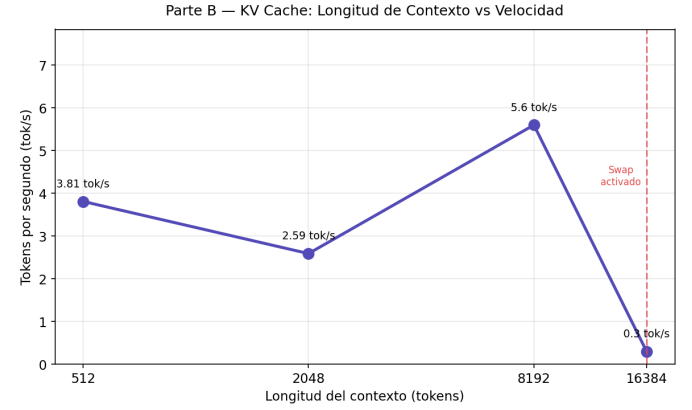


Fig. 3. Longitud de contexto vs velocidad de inferencia. La línea roja marca la activación del swap.

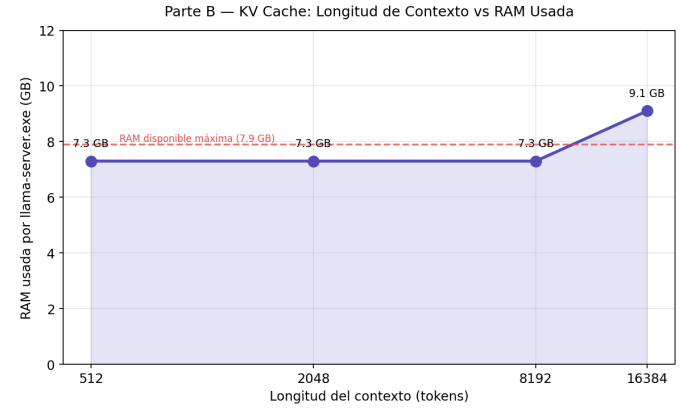


Fig. 4. Longitud de contexto vs RAM usada por el proceso de inferencia.

C. Parte C: Evaluación RAG

La Tabla IV compara las respuestas del modelo con y sin recuperación de contexto para cinco preguntas sobre el corpus.

El caso más ilustrativo fue Aldonza Lorenzo. Sin RAG el modelo respondió: “No tengo información sobre una persona llamada Aldonza Lorenzo relacionada con Don Quijote.” Con RAG el sistema recuperó el fragmento 11 del índice con similitud coseno de 0.597 y el modelo describió correctamente a la moza labradora, explicó el cambio de nombre a Dulcinea del Toboso y citó frases del texto original. El caso de los molinos

TABLE IV
COMPARATIVA RAG VS SIN RAG — CORPUS DON QUIJOTE

Pregunta	Sin RAG	Con RAG
Nombre del caballo	Inventa significado	Cita textual
Quién es Aldonza	Sin información	Explica Dulcinea
Molinos de viento	Inventa dama	Cita episodio real
Armas y caballo	Inventa físico	Cita descripción
Motivación caballero	Genérico	Cita reglas

de viento mostró la alucinación más llamativa sin RAG: el modelo inventó una “dama de Lorna”, figura mitológica que no existe en el libro ni en ningún contexto relacionado.

D. Parte D: Integración de herramientas

La Tabla V resume las seis herramientas integradas con sus características principales.

TABLE V
HERRAMIENTAS EXTERNAS INTEGRADAS EN JARVIS

Herramienta	API	Auth	Éxito
Clima actual	Open-Meteo	No	100%
Precio cripto	CoinGecko	No	100%
Info GitHub	GitHub	No	100%
Eventos hoy	Google Cal	OAuth	100%
Próximos eventos	Google Cal	OAuth	100%
Buscar eventos	Google Cal	OAuth	100%

Los fallos documentados son controlados y deliberados. Una ciudad fuera de la base de datos devuelve el mensaje “Ciudad no disponible” sin inventar temperatura ni condición climática. Un repositorio inexistente maneja el código HTTP 404 de GitHub y devuelve un mensaje de error claro sin alucinar datos de estrellas o forks. Una moneda inválida en CoinGecko devuelve el error de la API sin fabricar un precio numérico. En los tres casos el sistema falla con gracia en lugar de producir información falsa, lo que representa un comportamiento deseable desde la perspectiva de la confiabilidad del sistema.

E. Parte E: Evaluación automatizada

La Tabla VI resume los resultados del conjunto de 20 indicaciones ejecutadas de forma automática mediante `eval_runner.py`.

TABLE VI
RESULTADOS DE EVALUACIÓN AUTOMATIZADA POR CATEGORÍA

Categoría	N	Éxitos	Tasa	Lat.
Chat puro	4	3	75.0%	32.9 s
RAG requerido	4	3	75.0%	40.7 s
Herramienta	4	4	100.0%	17.4 s
Multistep	4	1	25.0%	13.6 s
Adversarial	4	2	50.0%	10.4 s
Global	20	13	65.0%	22.9 s

La categoría de herramienta individual alcanzó el 100% porque el modelo sigue con alta fiabilidad el formato JSON

cuando la tarea es directa y la herramienta correcta es inequívoca a partir del vocabulario de la pregunta. Las tareas multistep cayeron al 25% porque exigen que el modelo combine datos reales de una API con razonamiento adicional en una sola respuesta: en la práctica el modelo completa con frecuencia uno de los dos pasos esperados pero raramente ambos con la calidad mínima requerida por los criterios de evaluación. Las tareas adversariales alcanzaron el 50%: dos fallos se manejaron con gracia, pero dos indicaciones provocaron respuestas que no admitían explícitamente la limitación, lo que se clasificó como fallo según los criterios del conjunto de prueba.

IV. DISCUSIÓN

A. Comparación con LLMs en la nube

Se aplicaron cualitativamente los mismos prompts del conjunto de prueba a Claude Sonnet 4.6 para establecer una referencia comparativa. Las diferencias más marcadas se observaron en tres áreas concretas. En tool calling, el modelo de nube nunca generó JSON malformado ni seleccionó la herramienta incorrecta en ninguna de las 20 indicaciones, mientras que Jarvis falló en cuatro. En razonamiento encadenado, el modelo de nube completó correctamente las cuatro tareas multistep articulando tanto los datos reales como la interpretación derivada en una sola respuesta coherente y detallada. En el manejo de texto en español antiguo, el modelo de nube relacionó correctamente las preguntas contemporáneas con el vocabulario de Cervantes sin necesidad de un modelo de embeddings especializado ni de instrucciones adicionales en el prompt, lo que sugiere que la diferencia no es de arquitectura sino de escala de parámetros y volumen de datos de preentrenamiento.

B. Límites honestos del sistema

Las tareas multistep representan la limitación más clara de Jarvis con el hardware actual. La latencia promedio de las tareas que requieren herramienta es de 17.4 segundos por la doble llamada al modelo, y añadir un paso de razonamiento adicional sobre los datos recuperados eleva ese tiempo a un rango donde la experiencia de usuario se vuelve poco práctica para un asistente conversacional.

El corpus RAG con texto en español del siglo XVII presentó dificultades semánticas inesperadas durante el desarrollo. El modelo de embeddings `all-MiniLM-L6-v2`, entrenado principalmente en inglés moderno, no relacionaba correctamente las preguntas contemporáneas con el vocabulario de Cervantes: una pregunta sobre “el nombre del caballo de Don Quijote” no recuperaba el fragmento que contenía la palabra “Rocinante” porque la similitud coseno entre ambas representaciones era insuficiente. El problema se resolvió migrando a `paraphrase-multilingual-MiniLM-L12-v2`, que produjo recuperaciones semánticamente coherentes para todas las preguntas de prueba.

C. Mejoras propuestas

Con el doble de RAM disponible o una GPU de gama de entrada como una RTX 3060, dos mejoras serían directamente factibles. En primer lugar, cargar el modelo Q8_0 con aceleración GPU elevaría la velocidad de inferencia a un rango estimado de 80 a 100 tok/s, reduciendo la latencia de las tareas multistep a menos de 5 segundos y haciendo viable el razonamiento encadenado en tiempo real. En segundo lugar, la indexación del corpus completo de *Don Quijote* (aproximadamente 2.1 millones de caracteres divididos en cerca de 2,200 fragmentos) sería procesable en tiempos razonables, extendiendo la cobertura del RAG a la totalidad de la obra y mejorando la tasa de éxito de la categoría RAG requerido desde el 75% actual.

V. EXTENSIONES BONUS

Más allá de los requisitos base se implementaron cuatro extensiones. La documentación completa de cada una está disponible en el repositorio github.com/Andr3sss/jarvis.

Comparativa de familias de modelos. Se evaluó phi4-mini:3.8b de Microsoft bajo las mismas condiciones y rúbrica que Llama 3.2 3B. En Q4_K_M alcanzó 22.77 tok/s con calidad 2.6/3.0, comparable a Llama Q4_K_M (27.87 tok/s, 2.6/3.0). Phi-4-mini demostró mayor precisión en tareas de código pero menor robustez ante instrucciones de formato estricto, donde Llama mostró comportamiento más predecible.

Interfaz gráfica local con control por voz. Se implementó una interfaz web (Flask, localhost:5000) con selección manual o automática entre los 5 modelos disponibles, historial de conversaciones persistente, control por voz usando Web Speech API de Chrome para entrada y salida de audio, y un panel de gestión visual con formularios modales para notas, tareas y eventos del calendario.

Gestor local de notas y tareas. Se añadieron 9 herramientas para CRUD completo de notas, guardadas simultáneamente en JSON y como archivos .txt legibles, y de tareas con estado pendiente o completada. El orquestador secuencial permite combinar herramientas en un solo prompt: una indicación como “cuánto cuesta el bitcoin y guarda el precio como nota” ejecuta precio_cripto y crear_nota en secuencia automáticamente, resolviendo dependencias entre resultados intermedios.

Google Calendar con escritura completa. Se amplió el scope OAuth de solo lectura a escritura total, implementando creación, edición y eliminación de eventos. El sistema resuelve el ID del evento internamente mediante búsqueda previa por nombre, de modo que el usuario solo necesita indicar el título del evento para editarlo o eliminarlo sin conocer identificadores internos de Google.

VI. APÉNDICE DE REPRODUCIBILIDAD

El repositorio en github.com/Andr3sss/jarvis contiene todos los entregables. Los pasos esenciales para reproducir el sistema completo son:

```
git clone https://github.com/Andr3sss/jarvis
cd jarvis
pip install requests psutil matplotlib pandas \
    chromadb sentence-transformers langchain \
    langchain-community google-auth \
    google-auth-oauthlib google-auth-httplib2 \
    google-api-python-client flask flask-cors
ollama pull llama3.2:3b-instruct-q4_K_M
ollama pull llama3.2:3b-instruct-q8_0
ollama pull llama3.2:3b-instruct-q3_K_M
python rag/rag_pipeline.py --build
python benchmarks/benchmark_quant.py
python benchmarks/benchmark_kvcache.py
python eval/eval_runner.py
python jarvis_server.py
```

Las versiones principales utilizadas son chromadb 0.6.x, sentence-transformers 3.x y ollama 0.4.x. Los archivos credentials.json y token.json de Google Calendar no se incluyen en el repositorio por seguridad y deben generarse siguiendo las instrucciones del README.

REFERENCES

- [1] Meta AI, “Llama 3 Model Card,” Meta Platforms, Inc., 2024. [En línea]. Disponible en: <https://github.com/meta-llama/llama3>
- [2] T. Dettmers, A. Pagnoni, A. Holtzman y L. Zettlemoyer, “QLoRA: Efficient Finetuning of Quantized LLMs,” en *Advances in Neural Information Processing Systems*, vol. 36, 2023.
- [3] N. Reimers e I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” en *Proc. EMNLP 2019*, pp. 3982–3992, 2019.
- [4] P. Lewis *et al.*, “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” en *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459–9474, 2020.
- [5] A. Vaswani *et al.*, “Attention Is All You Need,” en *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [6] Chroma Research, “ChromaDB: The AI-native open-source embedding database,” 2024. [En línea]. Disponible en: <https://www.trychroma.com>
- [7] Google LLC, “Google Calendar API Documentation,” 2024. [En línea]. Disponible en: <https://developers.google.com/calendar>
- [8] Microsoft Research, “Phi-4-Mini Technical Report,” Microsoft, 2025. [En línea]. Disponible en: <https://azure.microsoft.com/en-us/blog/phi-4-mini>
- [9] A. Ronacher, “Flask: A lightweight WSGI web application framework,” Pallets Projects, 2024. [En línea]. Disponible en: <https://flask.palletsprojects.com>
- [10] W3C, “Web Speech API Specification,” World Wide Web Consortium, 2024. [En línea]. Disponible en: <https://wicg.github.io/speech-api>

Declaración de uso de IA: Este trabajo utilizó Claude Sonnet 4.6 (Anthropic) como asistente durante el desarrollo del código y la redacción del informe. Todas las mediciones, puntuaciones y decisiones de diseño fueron realizadas y verificadas por el autor en su propio hardware.